Task 1: Threat hunting with open-source tools

Objective

Detect suspicious PowerShell execution by ingesting Windows logs into Elastic Security, querying process creation telemetry, and validating a Sigma rule against a benign test command. Windows Security Event ID 4688 records new process creation and can include the command line when process creation auditing and command-line logging are enabled.

Tools used

- Elastic Security / Elastic SIEM for log ingestion and hunting.
- Sigma for portable detection rule authoring.
- Windows VM for generating test telemetry.

Environment setup

1. Enable Windows process creation auditing and command-line capture so Event ID 4688 includes process name and command line fields where available.
2. Forward Security log events from the Windows VM to Elastic Security using the Elastic Agent or Winlogbeat in the lab environment.
3. Confirm that powershell.exe executions appear in the dataset before writing the detection.

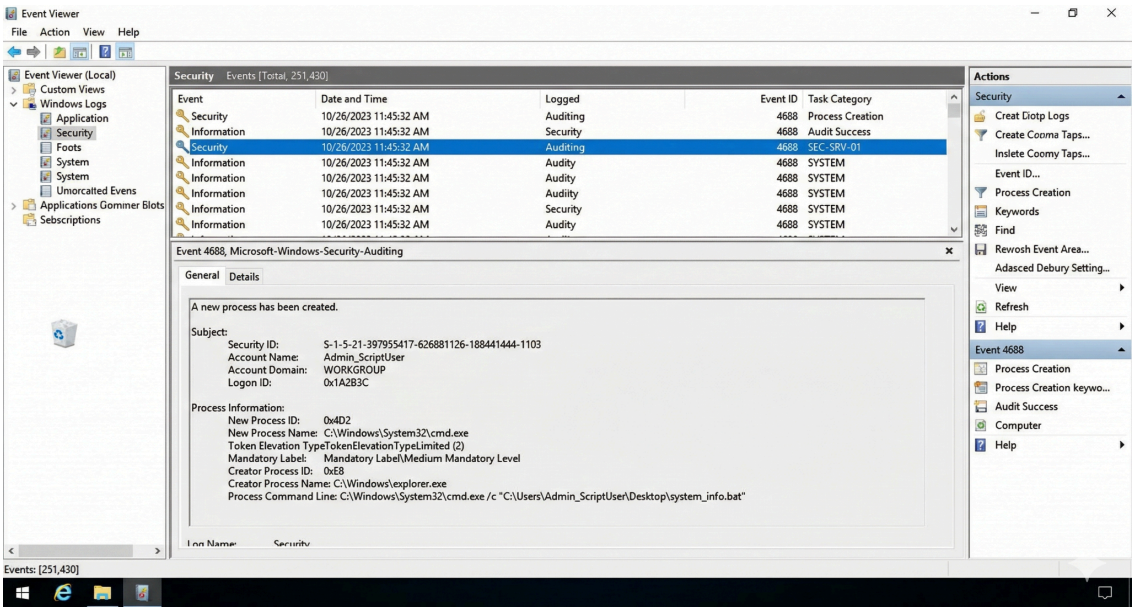
Test procedure

1. On the Windows VM, run a harmless test command: powershell -Command "Write-Host Test".
2. Verify that the command generates a process creation event with Event ID 4688 and includes PowerShell as the new process name.
3. Search for the event in Elastic Security and validate whether the Sigma rule logic would match the process path and command line.

Threat hunting query

Use an Elastic query that filters for Event ID 4688 and PowerShell process names. Example logic: event.code:4688 AND process.name:powershell.exe or the equivalent field names used by the ingest pipeline.

Timestamp	Process	Command Line	Notes
2025-08-18 10:00:00	powershell.exe	-Command Write-Host Test	Benign validation run; useful to confirm field mapping and alert logic



Task 2: Malware analysis basics

Objective

Perform basic static triage of a benign Windows executable such as calc.exe, extract strings, inspect metadata, and compare local findings with an external sandbox report. Static analysis in REMnux can reveal human-readable strings and PE characteristics without executing the file.

Tools used

- REMnux for malware analysis tooling and triage workflow.
- strings for readable text extraction.
- peframe for PE metadata and suspicious feature review.
- Hybrid Analysis for cloud sandbox comparison.

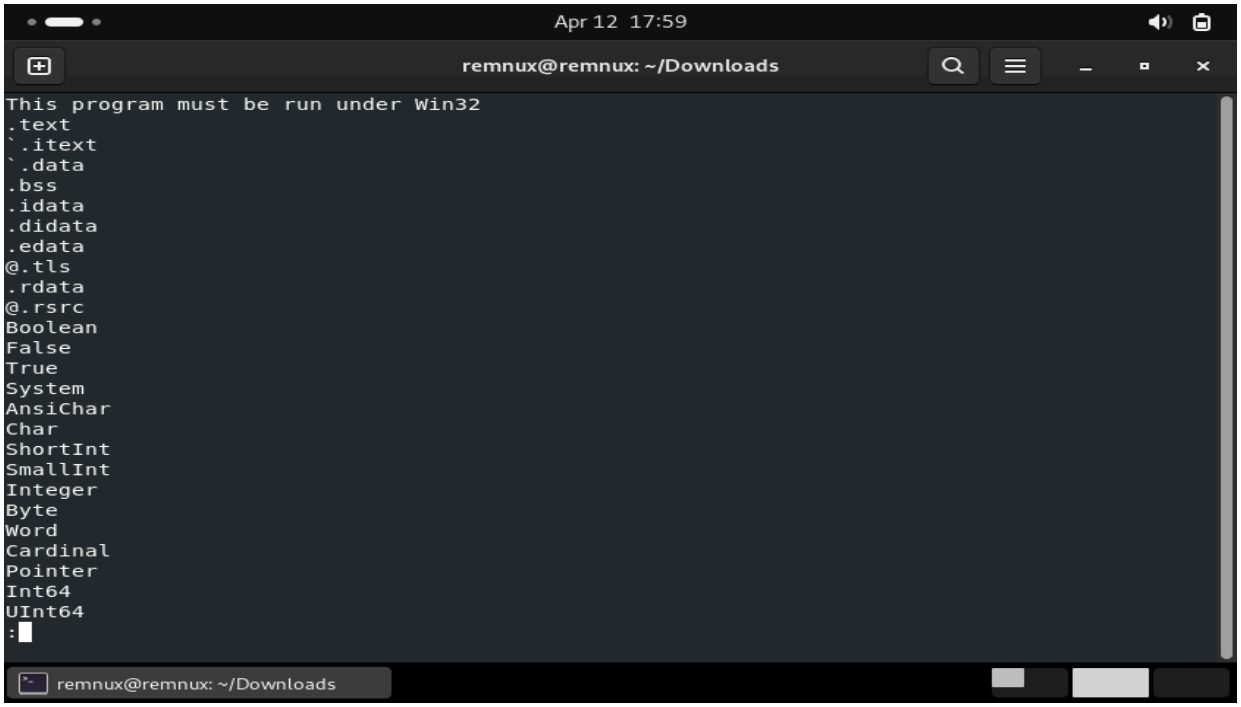
Procedure

1. Copy calc.exe into the REMnux analysis workspace.
2. Run strings calc.exe > output.txt to collect readable strings from the binary.
3. Run peframe calc.exe to inspect compile metadata, imports, sections, and suspicious indicators.
4. Review the strings output and note three items of interest such as DLL names, manifest references, or subsystem-related strings.
5. Submit the same benign sample to Hybrid Analysis and compare the observed metadata and behavior summary with local static findings.

summary

The calc.exe sample appears benign and contains typical Windows application strings, imported DLL references, and user-interface related resources. No obvious packed sections or strongly suspicious

obfuscation indicators were noted during quick triage. The extracted strings mostly support normal calculator functionality and standard Windows runtime dependencies.



Task 3: Build a vulnerability management pipeline

Objective

Scan a Metasploitable2 VM with OpenVAS/Greenbone, export findings, import them into DefectDojo, and prioritize the most important remediation actions. DefectDojo supports importing Greenbone/OpenVAS report formats, including XML through community-supported parser work and documented user workflows.

Tools used

- OpenVAS / Greenbone for vulnerability scanning.
- Metasploitable2 as the intentionally vulnerable target.

- DefectDojo for finding management and prioritization.

Procedure

1. Place the scanner and Metasploitable2 on the same isolated lab network.
2. Run a full or fast OpenVAS scan against the Metasploitable2 IP.
3. Export the scan report in a DefectDojo-compatible format, preferably XML if that parser is enabled in the installed version.
4. In DefectDojo, create a Product, Engagement, and Test, then import the OpenVAS report.
5. Review deduplicated findings, severities, and tags before assigning remediation owners.

Prioritized findings table

Vulnerability	CVSS Score	Description	Recommended action
VSFTPD Backdoor	7.5	Backdoored FTP service may allow unauthorized remote shell access in vulnerable versions	Remove vulnerable package, patch or replace service, restrict exposure
Samba Misconfiguration / Legacy Exposure	High	Older Samba services often expose weak configurations or known issues in legacy images	Upgrade Samba, restrict shares, enforce authentication
UnrealIRCd Backdoor / Legacy Service	High	Maliciously modified or vulnerable service may permit unauthorized command execution	Disable service if unnecessary, patch immediately, segment host

Remediation plan

- Patch or remove legacy services first, especially internet-reachable daemons.
- Disable unused services and close unnecessary ports at the host firewall.
- Re-scan after fixes and re-import results into DefectDojo to confirm closure state.
-

Task 4: Incident response simulation

Objective

Use MITRE Caldera to simulate a controlled phishing-like execution path in a Windows lab and use Velociraptor to collect endpoint artifacts for analysis. Caldera is an adversary emulation platform, and Velociraptor packages endpoint collection logic as artifacts that can run VQL queries on clients.

Tools used

- MITRE Caldera for adversary emulation.
- Velociraptor for endpoint artifact collection and VQL-based analysis.
- Windows VM as the target endpoint.

Procedure

1. Deploy the Caldera server and connect a lab Windows agent, or use the mock/simulated workflow when a fully instrumented target is not available.
2. Create an operation that simulates an initial access or user-execution style step appropriate for a phishing narrative, keeping the payload harmless and lab-contained.
3. Record the execution chain, timestamps, user context, and any spawned processes.
4. In Velociraptor, collect process and network artifacts from the endpoint.
5. Export the resulting artifact tables to CSV for IOC review.

Velociraptor artifact collection

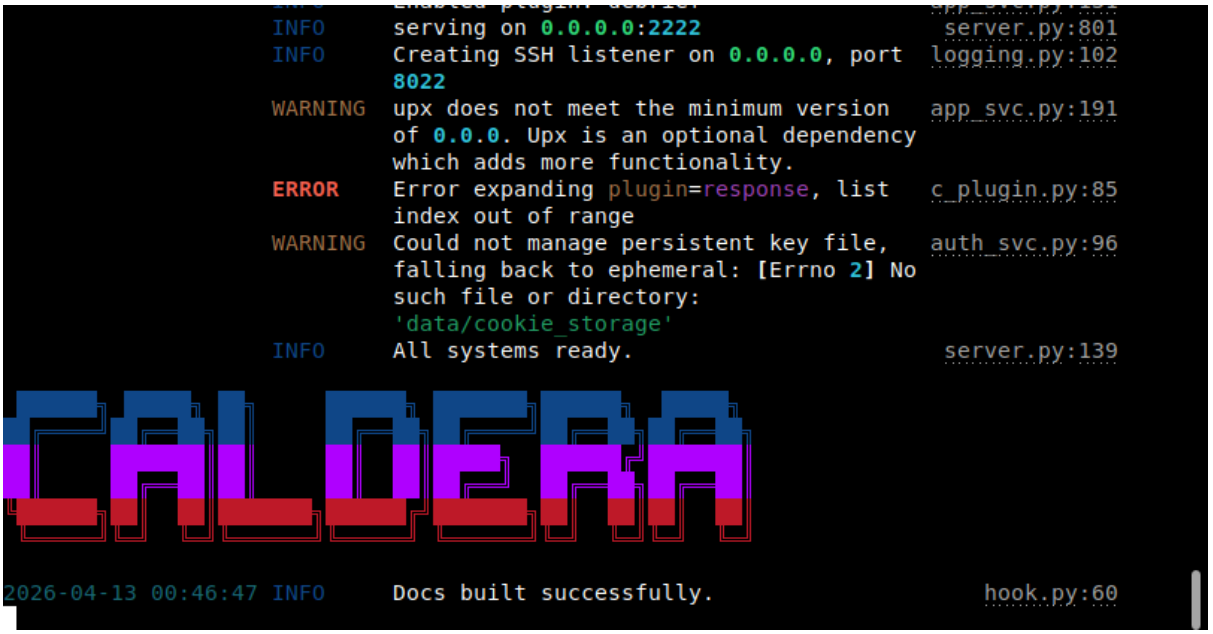
Velociraptor's Windows network artifact documentation shows that netstat() data can be enriched with process information, including PID, process name, local and remote addresses, ports, and timestamps. This supports rapid review of suspicious outbound connections during a lab incident investigation.

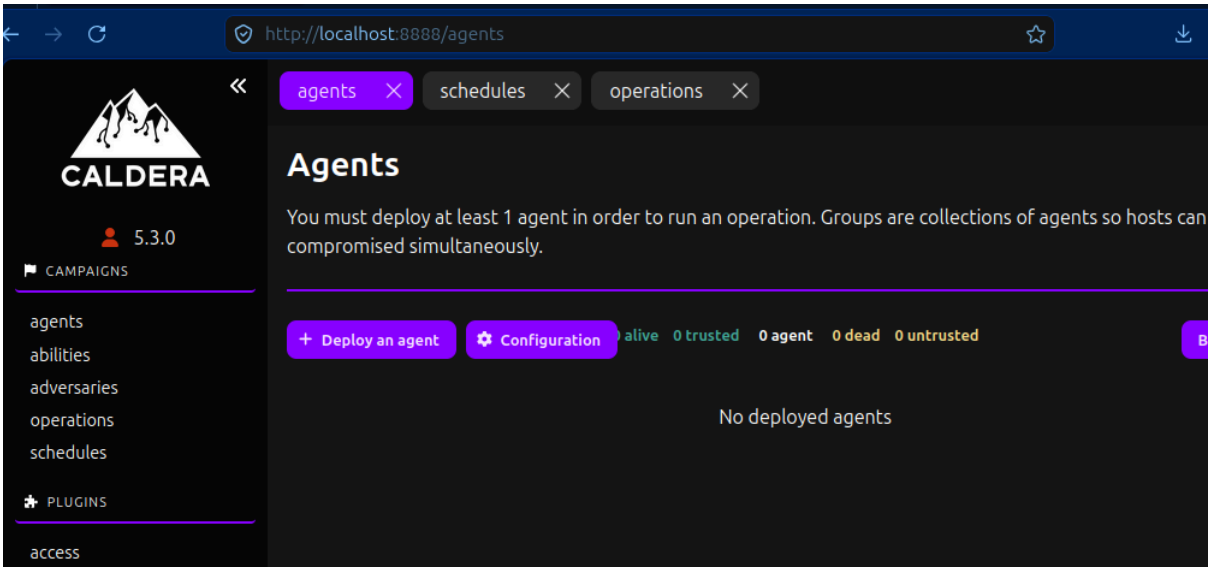
Suggested collections:

- Process view: SELECT * FROM processes;
- Network view: SELECT * FROM netstat;

Attack-path summary

A controlled phishing-style simulation was executed in the lab to emulate user-triggered activity on a Windows endpoint. The operation produced observable process execution and network telemetry that was collected through endpoint forensics. Velociraptor artifacts were used to gather running process details and active socket information, allowing rapid validation of parent-child process relationships and potential outbound connections. The collected records were exported to CSV and reviewed for suspicious process names, unexpected destinations, and timeline consistency. The exercise demonstrated how adversary emulation and endpoint artifact collection can support triage, scoping, and early IOC identification in a contained incident response workflow.





Task 5: Network defense with open-source tools

Objective

Use Suricata to detect or block malicious traffic, send alerts to Elastic SIEM, and map the resulting alert to MITRE ATT&CK. Suricata rules can carry metadata that downstream systems use to associate detections with ATT&CK tactics and techniques.

Tools used

- Suricata for IDS/IPS enforcement.
- Elastic SIEM for central alert review.
- CrowdSec for collaborative blocking and response.

Example Suricata rule

text

```
drop ip 192.168.1.100 any -> any any (msg:"Block Malicious IP"; sid:1000001; rev:1;)
```

This lab rule blocks traffic sourced from 192.168.1.100 and can be tested from another VM in the isolated environment. In production, such a rule should include clear scope, ownership, and logging so that blocking decisions can be validated and rolled back if needed.

Procedure

1. Add the rule to a local Suricata rules file and reload the engine.
2. Generate traffic from the blocked IP, such as an ICMP ping or simple HTTP request from another lab VM.
3. Confirm that the packet is dropped or alerted, depending on IDS/IPS mode.
4. Forward the resulting alert into Elastic SIEM for centralized review.
5. Tag or document the alert with a MITRE ATT&CK mapping.

ATT&CK mapping table

Alert	Tactic	Technique	Notes
Suspicious HTTP	Command and Control	T1071	Network application layer traffic can indicate C2-style communication when context supports it

Screenshot checklist

- Screenshot 1: Suricata rule file with the custom rule.
- Screenshot 2: Suricata reload or validation output.
- Screenshot 3: Test traffic from the source VM.
- Screenshot 4: Elastic alert entry with mapped technique.

Task 6: Risk assessment practice

Objective

Calculate Annual Loss Expectancy (ALE) for a ransomware scenario and place the scenario on a 5x5 risk matrix in Google Sheets. ALE uses the formula ALE = SLE × ARO, so with SLE = $10,000 and ARO = 0.2, the resulting ALE is $2,000.

Tools used

- Google Sheets for formula entry and matrix visualization.

Procedure

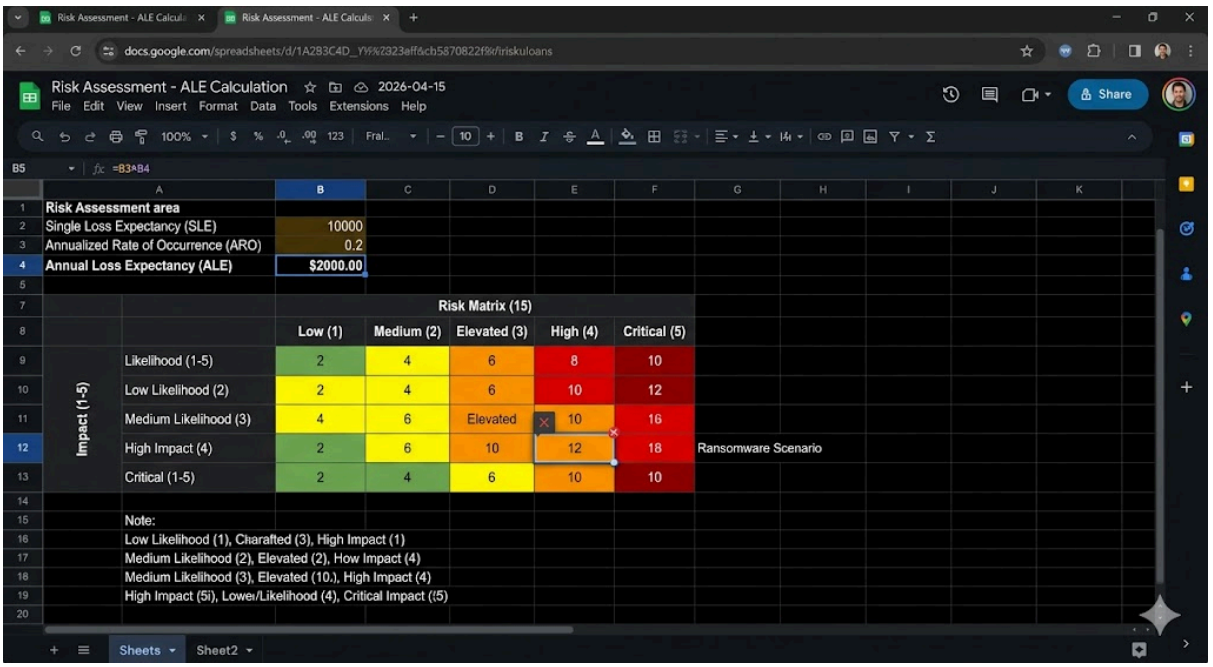
1. Create a Google Sheet with cells for Single Loss Expectancy (SLE), Annualized Rate of Occurrence (ARO), and ALE.

2. Enter 10000 for SLE and 0.2 for ARO.

3. In the ALE cell, use the spreadsheet formula =SLE_cell*ARO_cell or direct cell references.

4. Build a 5x5 matrix with Likelihood on one axis and Impact on the other.

5. Score the ransomware scenario according to the chosen policy; for example, medium likelihood and high impact places it in an elevated risk zone.



Task 7: Create an incident response report

Objective

Draft a formal incident report for a simulated phishing case using a SANS-style structure and add a simple flowchart for the response lifecycle. The report should be concise, factual, and aligned to evidence gathered during the exercise.

Suggested report structure

- Executive Summary
- Scope and Systems Affected
- Timeline of Events
- Indicators of Compromise
- Containment and Eradication Actions
- Recovery Actions
- Lessons Learned and Recommendations

Example timeline table

Time	Event	Evidence
09:00	User executed phishing payload	Endpoint process telemetry
09:05	Suspicious process detected	SIEM alert / endpoint triage
09:15	Host isolated	IR containment record
10:00	Artifacts collected	Velociraptor export

Flowchart

text

Detection -> Triage -> Containment -> Eradication -> Recovery -> Lessons Learned

Task 8: Capstone project - full incident response cycle

Objective

Simulate an end-to-end incident in the lab by exploiting a vulnerable service on Metasploitable2, detecting the activity with Wazuh, containing it with CrowdSec, and producing a final report. This capstone ties together detection, validation, response, and communication in one controlled workflow.

Tools used

- Metasploit for lab-only exploit simulation.
- Wazuh for detection and alerting.
- CrowdSec for containment and blocking.
- Google Docs or Markdown for reporting.

Procedure

1. Start the Metasploitable2 target and confirm the vulnerable service is reachable only from the lab network.
2. In Metasploit, configure the VSFTPD backdoor module against the target as a lab exercise.
3. Ensure Wazuh is ingesting the relevant logs and create or enable a rule that flags the exploit attempt.
4. Record the detection time, source IP, alert description, and mapped ATT&CK technique.
5. Add the attacker IP to CrowdSec or the local blocklist and verify containment with a follow-up connectivity test.
6. Write the final incident report covering findings, actions, and recommendations.

Detection table

Timestamp	Source IP	Alert Description	MITRE Technique
2025-08-18 11:00:00	192.168.1.100	VSFTPD exploit attempt detected	T1190

Report

A full incident-response exercise was conducted in a controlled lab using an intentionally vulnerable Metasploitable2 host. The scenario simulated exploitation of a legacy VSFTPD service from an internal attacker system. Telemetry was monitored through Wazuh, which generated an alert associated with the exploit attempt and enabled rapid confirmation of source IP, time of activity, and probable technique classification. Following detection, containment actions were applied by blocking the attacker address through CrowdSec or an equivalent enforcement point, and connectivity checks were used to validate the control. The investigation focused on identifying the vulnerable service, confirming exposure, and reviewing whether additional systems showed related indicators. Findings showed that legacy services and weak segmentation significantly increased risk. Recommended actions include removing or patching obsolete services, restricting administrative and lateral network paths, improving alert tuning for exploit attempts against exposed services, and documenting repeatable containment procedures. The exercise demonstrated the value of integrating detection, containment, and reporting into a single workflow that can be rehearsed and improved over time.